

Chapitre 5

TD Principes de base de Kotlin

v2.0

Plus sur les principes de base de Kotlin, la programmation orientée objet et les lambdas.

Sommaire

- Exercices:
 1. Notifications mobiles
 2. Prix du billet de cinéma
 3. Convertisseur de température
 4. Catalogue de chansons
 5. Profil Internet
 6. Téléphones pliables
 7. Enchère spéciale

S'entraîner: principes de base de Kotlin

Appliquez les concepts de base du langage de programmation Kotlin pour résoudre des problèmes spécifiques.

S'entraîner: principes de base de Kotlin

Notifications mobiles

- **Rédiger le code sur le playground Kotlin et partager le lien sur la plateforme.**
- En général, vous recevez un récapitulatif des notifications sur votre téléphone.
- Dans le code initial fourni dans l'extrait de code suivant, rédigez un programme qui affichera le message récapitulatif en fonction du nombre de notifications que vous avez reçues.
- Ce message devra inclure les éléments suivants :
 - Nombre exact de notifications lorsqu'il ne dépasse pas 100 notifications
 - 99+ pour le nombre de notifications lorsqu'il y en a plus de 100

S'entraîner: principes de base de Kotlin

Notifications mobiles

- ```
fun main() {
```
  - ```
    val morningNotification = 51
```
 - ```
 val eveningNotification = 135
```
  - ```
    printNotificationSummary(morningNotification)
```
 - ```
 printNotificationSummary(eveningNotification)
```
  - ```
}
```
 - ```
fun printNotificationSummary(numberOfMessages: Int) {
```
  - ```
    // Fill in the code.
```
 - ```
}
```
- Exécutez la fonction `printNotificationSummary()` pour que le programme affiche les lignes suivantes :

# S'entraîner: principes de base de Kotlin

## Notifications mobiles

- You have 51 notifications.
- Your phone is blowing up! You have 99+ notifications.

# S'entraîner: principes de base de Kotlin

## Prix du billet de cinéma

- **Rédiger le code sur le playground Kotlin et partager le lien sur la plateforme.**
- Les places de cinéma sont généralement facturées différemment selon l'âge des spectateurs.
- Dans le code initial fourni dans l'extrait de code suivant, rédigez un programme qui calculera le prix des billets en fonction de l'âge :
  - Un billet enfant de 15 \$ pour les personnes âgées de 12 ans et moins.
  - Un prix standard de 30 \$ par billet pour les personnes âgées de 13 à 60 ans. Le lundi, faites passer le prix du billet standard à 25 \$ pour cette tranche d'âge.
  - Un billet senior de 20 \$ pour les personnes âgées de 61 ans et plus. Nous supposons que l'âge maximal d'un spectateur est de 100 ans.

# S'entraîner: principes de base de Kotlin

## Prix du billet de cinéma

- Une valeur -1 pour indiquer que le prix n'est pas valable lorsqu'un utilisateur saisit une tranche d'âge qui ne correspond pas à celles spécifiées.
- ```
fun main() {
```
- ```
 val child = 5
```
- ```
    val adult = 28
```
- ```
 val senior = 87
```
- ```
    val isMonday = true
```
- ```
 println("The movie ticket price for a person aged $child is
 \${ticketPrice(child, isMonday)}.")
```
- ```
    println("The movie ticket price for a person aged $adult is  
    \${ticketPrice(adult, isMonday)}.")
```


S'entraîner: principes de base de Kotlin

Prix du billet de cinéma

- ```
println("The movie ticket price for a person aged $senior is\n${ticketPrice(senior, isMonday)}.")
```
- ```
}
```
- ```
fun ticketPrice(age: Int, isMonday: Boolean): Int {
```
- ```
    // Fill in the code.
```
- ```
}
```
- Exécutez la fonction `ticketPrice()` pour que le programme affiche les lignes suivantes :
  - The movie ticket price for a person aged 5 is \$15.
  - The movie ticket price for a person aged 28 is \$25.
  - The movie ticket price for a person aged 87 is \$20.

# S'entraîner: principes de base de Kotlin

## Convertisseur de température

- Il existe trois échelles de température principales : les degrés Celsius, les degrés Fahrenheit et le kelvin.
- Dans le code initial fourni dans l'extrait de code suivant, rédigez un programme qui convertira la température d'une échelle de température à une autre à l'aide des formules suivantes :
- Degrés Celsius à Fahrenheit :  $^{\circ} F = 9/5 (^{\circ} C) + 32$
- Kelvin à Celsius :  $^{\circ} = K - 273,15$
- Degrés Fahrenheit à kelvin :  $K = 5/9 (^{\circ}F - 32) + 273,15$
- Notez que la méthode `String.format("%.2f", /* measurement */)`  permet de convertir un nombre en type String avec deux décimales.

# S'entraîner: principes de base de Kotlin

## Convertisseur de température

- `fun main() {`
- `// Fill in the code.`
- `}`
- `fun printFinalTemperature(`
- `initialMeasurement: Double,`
- `initialUnit: String,`
- `finalUnit: String,`
- `conversionFormula: (Double) -> Double`
- `) {`
- `val finalMeasurement = String.format("%.2f",`  
`conversionFormula(initialMeasurement)) // two decimal places`

# S'entraîner: principes de base de Kotlin

## Convertisseur de température

- ```
println("$initialMeasurement degrees $initialUnit is $finalMeasurement  
degrees $finalUnit.")
```
- ```
}
```
- Exécutez la fonction `main()` pour qu'elle appelle la fonction `printFinalTemperature()` et affiche les lignes suivantes.
- Vous devrez transmettre des arguments pour la formule de température et de conversion.
- Conseil : Nous vous recommandons d'utiliser des valeurs `Double` pour éviter la troncature de l'entier (`Integer`) lors des opérations de division.

# S'entraîner: principes de base de Kotlin

## Convertisseur de température

- 27.0 degrees Celsius is 80.60 degrees Fahrenheit.
- 350.0 degrees Kelvin is 76.85 degrees Celsius.
- 10.0 degrees Fahrenheit is 260.93 degrees Kelvin.

# S'entraîner: principes de base de Kotlin

## Catalogue de chansons

- **Rédiger le code sur le playground Kotlin et partager le lien sur la plateforme.**
- Imaginez que vous deviez créer une application de lecture de musique.
- Créez une classe pouvant représenter la structure d'une chanson.
- La classe Song devra inclure les éléments de code suivants :
  - Propriétés de la chanson, artiste, année de publication et nombre de lectures
  - Propriété indiquant si la chanson est populaire (si le nombre de lectures est inférieur à 1 000, considérez-la comme peu populaire)
  - Méthode d'impression d'une description de chanson au format suivant :
    - "[Titre], interprété par [artiste], est sorti en [année de publication]."

# S'entraîner: principes de base de Kotlin

## Profil Internet

- **Rédiger le code sur le playground Kotlin et partager le lien sur la plateforme.**
- Il arrive souvent de devoir remplir des profils contenant des champs obligatoires et non obligatoires sur certains sites Web.
- Par exemple, vous pouvez ajouter vos informations personnelles et créer un lien vers les tiers qui vous ont conseillé de créer ce profil.
- Dans le code initial fourni dans l'extrait de code suivant, rédigez un programme qui affichera les détails du profil d'une personne.
  - ```
fun main() {
```
 - ```
 val amanda = Person("Amanda", 33, "play tennis", null)
```
  - ```
    val atiqah = Person("Atiqah", 28, "climb", amanda)
```
 - ```
 amanda.showProfile()
```

# S'entraîner: principes de base de Kotlin

## Profil Internet

- `atiqah.showProfile()`
- `}`
- `class Person(val name: String, val age: Int, val hobby: String?, val referrer: Person?) {`
- `fun showProfile() {`
- `// Fill in code`
- `}`
- `}`
- Exécutez la fonction `showProfile()` pour que le programme affiche les lignes suivantes :



# S'entraîner: principes de base de Kotlin

## Profil Internet

- Name: Amanda
  - Age: 33
  - Likes to play tennis. Doesn't have a referrer.
- 
- Name: Atiqah
  - Age: 28
  - Likes to climb. Has a referrer named Amanda, who likes to play tennis.

# S'entraîner: principes de base de Kotlin

## Téléphones pliables

- **Rédiger le code sur le playground Kotlin et partager le lien sur la plateforme.**
- En général, l'écran d'un téléphone s'allume et s'éteint lorsque l'utilisateur appuie sur le bouton Marche/Arrêt.
- En revanche, si un téléphone pliable est plié, son écran interne principal ne s'allume pas lorsque vous appuyez sur ce bouton.
- Dans le code initial fourni dans l'extrait de code suivant, écrivez une classe `FoldablePhone` qui héritera de la classe `Phone`.
  - Elle doit contenir les éléments suivants :
  - Une propriété qui indique si le téléphone est plié
  - Un comportement de la fonction `switchOn()` différent de celui de la classe `Phone` pour que l'écran ne s'allume que lorsque le téléphone n'est pas plié
  - Des méthodes permettant de modifier l'état du pliage

# S'entraîner: principes de base de Kotlin

## Téléphones pliables

- `class Phone(var isScreenLightOn: Boolean = false){`
- `fun switchOn() {`
- `isScreenLightOn = true`
- `}`
- `fun switchOff() {`
- `isScreenLightOn = false`
- `}`
- `fun checkPhoneScreenLight() {`
- `val phoneScreenLight = if (isScreenLightOn) "on" else "off"`
- `println("The phone screen's light is $phoneScreenLight.")`
- `}`
- `}`

# S'entraîner: principes de base de Kotlin

## Enchère spéciale

- **Rédiger le code sur le playground Kotlin et partager le lien sur la plateforme.**
- Lors d'une mise aux enchères, l'enchère la plus élevée détermine généralement le prix d'un article.
- Dans cette enchère spéciale, si personne n'enchérit pour un objet, celui-ci est automatiquement cédé à l'hôtel des ventes au prix minimum.
- Dans le code initial fourni dans l'extrait de code suivant, une fonction `auctionPrice()` (qui accepte un type `Bid?` pouvant avoir une valeur nulle en tant qu'argument) vous est fournie :
  - `fun main() {`
  - `val winningBid = Bid(5000, "Private Collector")`
  - `println("Item A is sold at ${auctionPrice(winningBid, 2000)}.")`

# S'entraîner: principes de base de Kotlin

## Enchère spéciale

- `println("Item B is sold at ${auctionPrice(null, 3000)}.")`
- `}`
- `class Bid(val amount: Int, val bidder: String)`
- `fun auctionPrice(bid: Bid?, minimumPrice: Int): Int {`
- `// Fill in the code.`
- `}`
- Exécutez la fonction `auctionPrice()` pour que le programme affiche les lignes suivantes :
  - `Item A is sold at 5000.`
  - `Item B is sold at 3000.`

# S'entraîner: principes de base de Kotlin

## Enchère spéciale

- `println("Item B is sold at ${auctionPrice(null, 3000)}.")`
- `}`
- `class Bid(val amount: Int, val bidder: String)`
- `fun auctionPrice(bid: Bid?, minimumPrice: Int): Int {`
- `// Fill in the code.`
- `}`
- Exécutez la fonction `auctionPrice()` pour que le programme affiche les lignes suivantes :
  - `Item A is sold at 5000.`
  - `Item B is sold at 3000.`